Project Specification: Personal Finance Tracker API

A small FastAPI + SQLAlchemy project designed to test the concepts learned so far. This document is a specification only; implementation code is intentionally omitted.

1. Project Goal

Build a REST API for tracking personal expenses. Demonstrate FastAPI routing, Pydantic request/response schemas, SQLAlchemy ORM, relationships, foreign keys, constraints, validation, transactions, error handling, and cascade deletion.

2. Scope

Use exactly two database entities:

```
User → Expense
```

Each Expense belongs to exactly one User. Deleting a User must automatically delete that user's Expenses.

3. User Entity

Field	Requirement
id	Primary key
name	Required string
email	Required; valid email; unique
created_at	Automatically generated timestamp

4. Expense Entity

Field	Requirement
id	Primary key
user_id	Foreign key to User; required
amount	Required; must be greater than 0
category	Required enum value
description	Required string
expense_date	Required date; cannot be in the future
created_at	Automatically generated timestamp

5. Expense Categories

Allowed values:

```
Food
Transport
Rent
Entertainment
Shopping
Other
```

6. API Endpoints

Method	Path	Purpose

POST	/users	Create a user
GET	/users/{user_id}	Get one user
DELETE	/users/{user_id}	Delete a user and cascade expenses
POST	/expenses	Create an expense
GET	/expenses	Get all expenses
GET	/expenses/{expense_id}	Get one expense
PUT	/expenses/{expense_id}	Update an expense
DELETE	/expenses/{expense_id}	Delete an expense

7. Required Schemas

Create request and response schemas for User and Expense. Response schemas should use from_attributes=True so SQLAlchemy ORM objects can be returned through Pydantic.

```
Suggested names: UserCreate, UserResponse, ExpenseCreate, ExpenseResponse.
```

8. Validation Rules

- User email must be a valid email address.
- User email must be unique.
- Expense user_id must reference an existing user.
- Expense amount must be greater than 0.
- Expense category must be one of the defined enum values.
- Expense date must not be in the future.
- Deleting a nonexistent resource should return 404.
- Updating a nonexistent expense should return 404.

9. Database Constraints & Relationships

Define a User → Expense one-to-many relationship. Expense.user_id must be a foreign key to User.id. Configure cascade deletion so deleting a user removes that user's expenses. The unique email rule must be enforced at the database level.

10. Error Handling Requirements

Use HTTPException for expected API errors. Missing users/expenses should produce 404 responses. Duplicate email should be handled cleanly as a 409 conflict, with a database rollback after an IntegrityError.

11. Testing Checklist

- Create a valid user.
- Create a user with an invalid email → 422.
- Create a duplicate email → 409.
- Get an existing user.
- Get a nonexistent user → 404.
- Create an expense for an existing user.
- Create an expense with a nonexistent user_id → 404.
- Create an expense with amount <= 0 → validation error.
- Create an expense with an invalid category → 422.
- Create an expense with a future date → validation error.

• Get all expenses.

• Get one expense.

• Update an existing expense.

• Update a nonexistent expense → 404.

• Delete an expense.

• Delete a nonexistent expense → 404.

• Delete a user who owns expenses and verify the expenses are automatically deleted.

12. Implementation Constraints

Do not add authentication, JWT, routers, async endpoints, frontend templates, pagination, or unrelated features. The goal is to test the concepts learned so far without creating another large project.

Build the project independently from this specification. The specification intentionally does not provide model, schema, route, or validator implementations.

13. Completion Criteria

The project is complete when all endpoints work, validation rules are enforced, database constraints are present, duplicate email is handled without an unhandled 500, cascade deletion is verified, and the testing checklist passes.